

ReVISE: Disrupting Industrial Control Processes with Timing-Aware PLC Variable Manipulation

Syed Ghazanfar Abbas¹, Muslum Ozgur Ozmen², Dave (Jing) Tian¹,
Z. Berkay Celik¹, Ashish Kundu³, Dongyan Xu¹

¹Purdue University, West Lafayette, USA

²Arizona State University, Tempe, USA

³Cisco Research, USA

{abbas4, daveti, zcelik, dxu}@purdue.edu, moozmen@asu.edu, ashkundu@cisco.com

Abstract—A trend in industrial control systems is that programmable logic controllers (PLCs) are increasingly becoming multi-app platforms, with industrial server programs running alongside the main PLC program. In this paper, we reveal that these programs access the same set of PLC variables without concurrency control. Consequently, we identify a class of PLC attacks where a remote attacker manipulates the sensing or actuation variables within a PLC cycle, via legitimate client-side APIs provided by the server applications *without compromising any program in the PLC*. Moreover, we observe that such attacks require *timing-awareness* for the variable manipulation – relative to the periodic execution of the main PLC program, which may invalidate the manipulation effect if the timing of the manipulation is not right. As the first effort to address the timing-awareness problem, we introduce ReVISE, an automated attack technique and a penetration-testing tool, which infers appropriate timing to manipulate target PLC variables with a significantly improved attack success rate. The input of ReVISE is a variable manipulation goal; and ReVISE will automatically achieve the goal via a three-step operation, including (1) PLC variable value monitoring; (2) timing-aware target variable manipulation; and (3) target variable value restoration. We evaluate ReVISE on three ICS experimental platforms controlled by real-world PLCs. ReVISE achieves an average of 86% success rate in 15 remote PLC variable manipulation attacks, compared to an average of 6% attack success rate without ReVISE.

Index Terms—Industrial Control Systems, Programmable Logic Controllers, ICS Security, Variable Manipulation

I. INTRODUCTION

The PLC is a special type of real-time computer that is widely used in process control in industrial control systems (ICS) [1]. Enforcing the process control loop, the PLC program execution is typically periodic, reflecting the sensing, control, and actuation of the control loop, with its state represented by corresponding sensing and actuation variables. In each PLC cycle (called *scan cycle*), (1) the sensing variables are updated with real-time sensor readings, (2) the main PLC program reads the sensing variables to compute the control output, which is written to the actuation variables, and (3) the PLC transmits the actuation variable values to the actuators [2].

Meanwhile, the PLC is increasingly becoming a multi-app platform, with embedded server applications such as OPC UA [3] and embedded web servers running side-by-side with the main PLC program for purposes such as data collection,

operation monitoring, and human-machine interaction. With no or weak authentication for the PLC device and the ICS network, it is possible for a remote attacker to connect to one of the on-PLC server applications from any device [4], [5]. In such a setting, a common attack strategy is *PLC variable manipulation*, as evidenced by real-world incidents, e.g., Stuxnet [6] and the Florida water treatment plant attack [7]. Previous research has proposed methods to determine which variables to manipulate and their target values [8]–[12]. Moreover, prior research has shown that attackers can inject malicious variable manipulation code into the main PLC program [6], [13] or firmware [14]. Recent research also shows that it is possible to first compromise the web server embedded in the PLC, and then remotely supply malicious values of the target PLC variables as input to the compromised web server [15].

In this paper, we extend and generalize this threat vector by revealing a class of attacks that can remotely manipulate sensing and actuation variables during a PLC scan cycle, through any embedded server program (not just the web server) and *without compromising any program in the PLC*. This class of attacks relies on our new findings about the embedded server programs: First, these programs run alongside the main PLC program and *share the same set of PLC variables*. Second, PLCs *lack concurrency control* over these variables, allowing server programs to access and modify the same variables during the same PLC cycle of the main PLC program. We observe that such a lack of concurrency control is common in commodity PLCs (e.g., Siemens and Codesys, which account for 56% of the PLC market [16]).

Despite the feasibility of *intra-cycle* variable manipulation, we are the first to identify and address the following challenge: such manipulations usually result in a low success rate in disrupting the target industrial control process, because of their lack of timing-awareness relative to the main PLC program's execution. More specifically, for a manipulated variable to be “consumed” by the main PLC program, the manipulation should happen *after* its correct value is written to the variable and *before* the main PLC program reads it in a PLC cycle. As an example, Figure 1 illustrates PLC execution during a cycle. In the cycle: (1) the sensing variables S_1 and S_2 are updated with real-time sensor readings, (2) the main PLC program

calculates the actuation variables A_1 and A_2 based on S_1 and S_2 , respectively, and (3) the actuation variables A_1 and A_2 are sent to the actuators. If an adversary manipulates variable S_1 , the manipulation will be effective only if it is done between the setting of S_1 and the reading of S_1 for computing A_1 (i.e., time window 2 in Figure 1). If the manipulation of S_1 happens in time window 1, it is too early, as the value will be overwritten by the (correct) sensor input. If the manipulation occurs in time window 3, it will be too late, as the value will no longer be accessed by the main PLC program during the same cycle.

In this paper, we present an automated attack technique and a penetration-testing tool, called ReVISE¹, that (1) significantly increases the success rate of the remote variable manipulation, and (2) enables controlled testing of intra-cycle variable-manipulation attacks on PLCs. Unlike previous work, ReVISE does not require compromising or injecting code into any program running in the PLC. ReVISE enables *timing-aware* PLC variable manipulation by first monitoring and then modifying the value of the target variable at a time favorable to the success of the manipulation. All these are performed by ReVISE remotely from the adversary’s attack device, via a legitimate API provided by any (benign) server program running in the PLC. The adversary only needs to set a manipulation goal (the “what”), generated by ReVISE using existing attack planning tools [9], [10] and ReVISE will automatically realize the goal via a three-step operation (the “how”), including (1) target variable value monitoring; (2) timing-aware target variable value modification; and (3) target variable value restoration (to original value).

More specifically, given a PLC variable manipulation goal, ReVISE first passively monitors the target PLC variable via a benign server program that runs concurrently with the main PLC program. Based on the variable values observed, ReVISE dynamically decides when to modify the target variable’s value, in synchronization with the main PLC program’s access to those variables, to increase the chances that the effect of the variable manipulation can be successfully inflicted upon the target ICS process. Lastly, ReVISE restores the target variable’s original value, after the main PLC program has “consumed” the manipulated value, to maintain stealth.

We have conducted 15 PLC variable manipulation attacks using ReVISE, across three ICS testbeds with *real* PLCs in the loop: (1) a Fischertechnik manufacturing model plant [17] with Siemens PLC, a simulated chemical plant [18] with Unipi PLC, and a simulated water treatment plant [19] with CodeSys PLC. ReVISE achieves an average of 86% success rate for the 15 attacks, compared with a 6% success rate when the attacks are launched without ReVISE. In the face of two state-of-the-art PLC attack detection systems, ReVISE-enabled attacks can evade detection with high probability. In summary, the contributions of this paper are as follows:

- We reveal a new class of PLC variable manipulation

¹The acronym REVISE stands for “Remote Variable Interference via Synchronized Execution”.

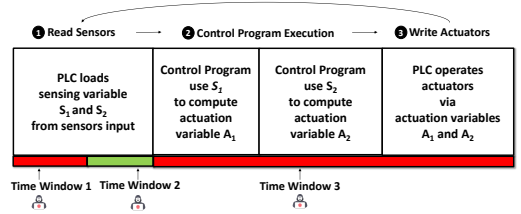


Fig. 1: Illustration of finding the correct *timing* to manipulate the sensing variable (S_1) during a PLC scan cycle.

attacks, based on our new findings about the PLC variables (i.e., being accessible without concurrency control to the main PLC program and the embedded server programs) (Section III). Such attacks are launched remotely via legitimate variable-read/write APIs and do not require compromising any program in the PLC.

- We are the first to identify and address the timing (un)awareness challenge in making remote variable manipulation successful (Section IV), based on the observation that previous PLC variable manipulation attacks suffer from a low success rate due to their lack of synchronization with the main PLC program execution.
- We develop ReVISE, an automated, penetration-testing tool for intra-cycle variable-manipulation attacks on PLCs (Section V). We show ReVISE’s effectiveness – with respect to the success rate and detection rate – on three ICS testbeds with three respective real-world PLCs in the loop.

ReVISE source code: <https://github.com/purseclab/ReVISE>.

II. THREAT MODEL

PLC and ICS Network. Our model reflects a typical ICS environment [2], [20]–[22], shown in Figure 2. We assume that (1) the sensors and actuators are trusted; (2) the PLC hardware and runtime are trusted; (3) the main PLC program and server programs running on top of the PLC runtime *cannot* be compromised or modified; (4) the main PLC program does not allow remote access, but the server programs allow remote access with no or weak client authentication; and (5) the ICS network does not have strong authentication, making it possible to connect to the PLC from the attacker’s own device [5], [23]–[25]. In fact, researchers have identified more than 36,700 PLCs that can be remotely accessed from the Internet [26].

Attacker’s Goal. The attacker’s goal is to remotely manipulate PLC variables to inflict physical disruption on the ICS process controlled by the PLC. For example, the attacker may modify a sensor variable value to mislead the PLC to make a certain control decision or directly modify an actuation variable to overwrite the control decision.

Attacker’s Capabilities. Without physical access to the PLC, its sensors or actuators, and without compromising any program on the PLC, the attacker can remotely access the server programs running in the PLC (only) via the legitimate APIs of those programs. We assume that, without in-depth knowledge about the target ICS, the attacker can leverage existing ICS

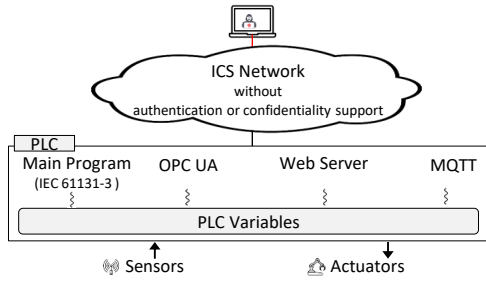


Fig. 2: A typical ICS environment on which our threat model is based. A remote PLC variable manipulation attack can be launched from the attacker’s device.

attack discovery/planning tools to automatically determine the PLC variable manipulation goal for each attack [9]–[12]. Such assumptions are consistent with real-world ICS attacks, such as Stuxnet [6], water treatment utilities in the US [7], [27].

III. A CLASS OF PLC ATTACKS VIA INTRA-CYCLE VARIABLE MANIPULATION

In this paper, we elevate and generalize the threat vector of PLC variable manipulation [13]–[15], [28] by revealing a class of remote intra-cycle variable manipulation attacks through *any* embedded server program and *without* compromising any program in the PLC. Our study of the 14 leading commodity PLCs, such as Siemens, CodeSys, Unipi, Rockwell Automation, Schneider Electric, and Mitsubishi, reveals a growing trend of integrating programs such as the OPC UA server, MQTT, web server, database connector, and cloud/edge agent into PLCs. These programs enable a wide range of remote operations important to industrial automation, such as monitoring, management, data analytics, and remote human-machine interaction.

By exploiting the weaker protection of ICS networks (compared with enterprise networks) [23], [29], an adversary can connect their device to one of the on-PLC server programs and remotely manipulate PLC variables via the legitimate API provided by the server program. This class of remote PLC variable manipulation attacks is enabled by two of our new findings about server programs embedded in PLCs: (1) common access to PLC variables, which are also accessed by the main PLC program, and (2) lack of concurrency control among all programs accessing the PLC variables.²

A. Common Access to PLC Variables

We first observe that server programs in PLCs have common access to PLC variables *by default*, just like the main PLC program. As shown in Figure 2, the PLC runtime is a multi-threaded process that manages the main PLC program, the server programs, and the PLC variables shared by all programs. At startup, the PLC runtime initializes the variables and runs the main PLC program and server programs as separate threads. These programs share the process memory space, including

the PLC variables. This allows a remote attacker to influence the main PLC program – without direct interaction with it. Instead, the attacker does that by modifying the PLC variable values via a server program, which by design allows remote client access. As a concrete example, the attacker can remotely access the OPC UA server in a PLC (e.g., Siemens, Codesys, and UniPi) to read/write *sensing* and *actuation* variables.

Locating PLC variables based on relative addresses. We have also found that an in-PLC server program can accurately locate those PLC variables by their relative addresses, which is on the contrary prohibited in the main PLC program. For example, Siemens’ main PLC program does not allow PLC variable access via relative addressing. Unfortunately, other server programs (e.g., the embedded web server) do not enforce such a restriction. Instead, they allow access to all PLC variables via their relative locations (indexes), providing unrestricted read and write access to all variables even without knowing the specific variable names.

B. Lack of Concurrency Control

We further discover a (surprising) lack of concurrency control over PLC variable access between the main PLC program and embedded server programs. This allows a server program to read and modify PLC variables, while the main PLC program is doing the same in each scan cycle. An attacker can exploit this deficiency by manipulating a sensing variable before the main PLC program accesses it to execute its control algorithm, or by manipulating an actuation variable after the main PLC program has computed the actuation output, leading to the attacker’s intended impact on the industrial process being controlled.

Tracking PLC variable value change during a scan cycle.

We identify one (legitimate) feature for remote PLC variable access that is particularly helpful to variable manipulation attacks: reading the same variable multiple times in *one* client request. This feature allows the attacker to monitor possible changes in the value of the variable during a PLC cycle. Our experiments with the OPC UA server and web server in Siemens, Codesys, and UniPi PLCs confirm this feature. A client request with duplicate reads of the same variable (V_1, V_1) to the server program may come back with a response showing a value change ($V_1 = 10, V_1 = 11$), which may, as to be leveraged in Section V, indicate a good timing for launching variable manipulation.

Multiple reads of the same variable in one client request help reduce both the number of requests and end-to-end latency for the attacker. In our experiment, two consecutive read requests for one variable (V_1) result in an end-to-end latency of 2ms, whereas the latency is reduced to 1ms if the reads are requested in one request. The reduced latency helps to catch the right timing for the subsequent manipulation of V_1 , hence increasing the probability of attack’s success.

IV. TIMING CHALLENGE FOR PLC VARIABLE MANIPULATION ATTACKS

Remote PLC variable manipulation attacks, enabled by our new findings in the previous section, pose a serious threat

²We have followed responsible disclosure practice and reported our findings to the PLC vendors.

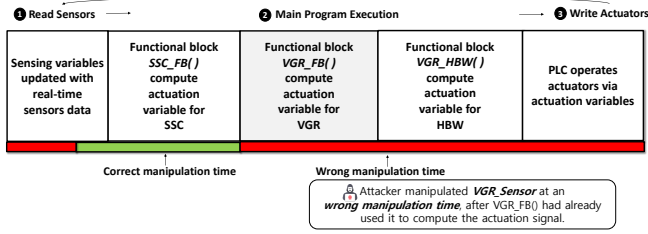


Fig. 3: Sensing variable manipulation during a Siemens PLC scan cycle controlling a Fischertechnik plant.

to ICS environments. However, we identify a key challenge in making PLC variable manipulation practical and effective. More specifically, if a variable manipulation is not performed in synchronization with the execution of the main PLC program during a PLC scan cycle, the corrupted variable value may not be “consumed” by the main PLC program, resulting in attack failure. We now use two sample attacks to illustrate this timing challenge.

Sample Attack 1: Manipulating a PLC sensing variable.

The Fischertechnik mini-plant [17] consists of typical industrial plant components, such as a vacuum gripper robot (VGR) and a high-bay warehouse (HBW). The mini-plant is controlled by a real Siemens PLC, where the main PLC program is structured into functional blocks, as shown in Figure 3, with each block managing a specific plant component (e.g., VGR_FB for controlling the VGR and HBW_FB for the HBW).

At the start of a PLC cycle, each functional block operates based on its respective sensing variables. For example, VGR_FB monitors the sensing variable (*VGR_Sensor*) to control the movement of the VGR robot. Specifically, if *VGR_Sensor* is within the range of [0, 600], VGR_FB sets the actuation signal *VGR_Actuator* to *True*, allowing the VGR robot to continue its movement toward the HBW station. When *VGR_Sensor* reaches the range of [601, 610], VGR_FB stops the VGR robot by setting *VGR_Actuator* to *False*. This ensures safe plant operation so that the VGR robot drops the workpiece precisely at the HBW station.

Attack Objective. The attacker’s objective is: when the value of *VGR_Sensor* is 80, change it to 601, to cause the VGR robot to stop at a maliciously intended position and drop the workpiece to an unsafe location.

Attack Result. Without synchronizing with the main PLC program execution, the attacker may not catch the right timing to change the value of *VGR_Sensor* during the PLC cycle, resulting in an unsuccessful attack. As shown in Figure 3, the attacker changes *VGR_Sensor* to 601, *after* function VGR_FB has processed the original (benign) value of *VGR_Sensor* (= 80) to compute the actuation signal *VGR_Actuator* (= *True*). As a result, the VGR robot continues its normal operation. In the next PLC cycle, the value of *VGR_Sensor* will be restored with the sensor’s new reading.

As shown in the first row of Table I, the manipulation of *VGR_Sensor* yields a low success rate: Out of 100 runs of

TABLE I: Success rate of sample attacks

Sample Attack	Variable Manipulated	No. of Tries	Success Rate
1	Sensing variable <i>VGR_Sensor</i>	100	6%
2	Actuation variable P203	100	8%

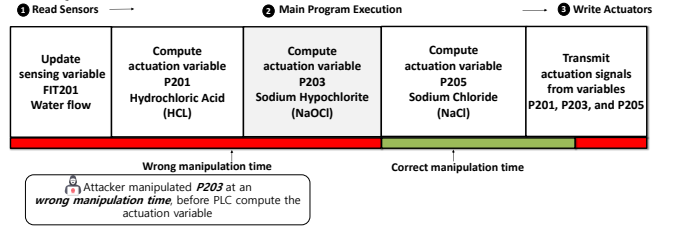


Fig. 4: Actuation variable manipulation during a CodeSys PLC scan cycle controlling a water treatment plant

the attack, only 6 succeeded, resulting in a success rate of 6%. In these successful runs, the manipulation of *VGR_Sensor* happened in the “green” time window in Figure 3, namely after *VGR_Sensor* had been updated with sensor reading and before the function VGR_FB used its value to compute the actuation signal.

Sample Attack 2: Manipulating a PLC actuation variable.

The water treatment plant [19] uses three distinct chemical storage tanks to treat raw water. These include a tank for hydrochloric acid (HCl), a tank for sodium hypochlorite (NaOCl), and a tank for sodium chloride (NaCl). These chemicals serve crucial functions, such as pH adjustment and disinfection. The main PLC program regulates the dosing of these chemicals through dedicated pump valves, each controlled by an actuation variable: variable P201 for the HCl pump valve, P203 for the NaOCl pump valve, and P205 for the NaCl pump valve. The main PLC program computes valve operations in real time based on the water flow rate measured by the sensing variable FIT201.

Attack Objective. The attacker aims to manipulate the value of actuation variable P203, increasing its value from 0.10 (normal) to 0.20, resulting in an increased NaOCl level in the water, hence serious water pollution.

Attack Result. As shown in Figure 4, without synchronizing with the main PLC program execution, the attacker may manipulate actuation variable P203, before the PLC computes the value of P203 based on its control algorithm. As a result, the malicious value (0.20) of P203 will be overwritten by the normal value (0.10) once the computation is finished, rendering the attack unsuccessful. As shown in the second row of Table I, the success rate of 100 tries of this attack is only 8%. For the 8 successful runs, the manipulation happened in one of the two “green” time windows, namely after the main PLC program had computed the value of P203 and before the value was transmitted to the actuator.

V. REVISE: TIMING-AWARE REMOTE PLC VARIABLE MANIPULATION

To address the variable manipulation *timing* challenge, identified in the previous section, we develop ReVISE, a

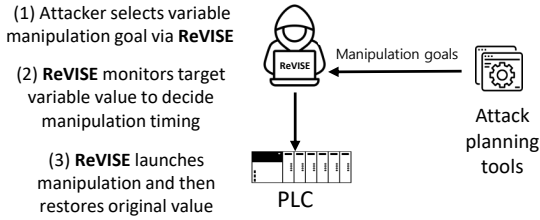


Fig. 5: End-to-end ReVISE-enabled variable manipulation.

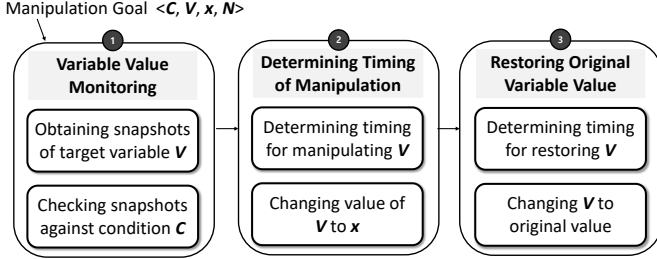


Fig. 6: Overview of ReVISE workflow.

generic, automated penetration-testing tool that enables timing-aware remote PLC variable manipulation. ReVISE needs to address the following key technical problems.

Remote estimate of variable manipulation timing. Being remote and not directly interacting with the main PLC program, the attacker cannot track the main PLC program’s execution in each cycle, hence cannot precisely identify the exact time window for manipulating a PLC variable. The end-to-end latency between the PLC and the attacker’s device adds to the difficulty of this problem. We resort to remotely polling the target PLC variable and estimating the right manipulation timing based on the polling result, with the goal of *probabilistically* improving the manipulation’s success rate.

Preserving stealthiness of variable manipulation. The manipulated value of the target PLC variable may lead to the detection of the attack. For example, the value may appear on the human machine interface (HMI) of the ICS and draw suspicion. To minimize the probability that a manipulated variable value is “seen” by a monitor or anomaly detector, we let ReVISE restore the original value of the manipulated variable, after the main PLC program has consumed the manipulated value.

A. ReVISE Overview

As an automated attack-testing tool, ReVISE can be used as shown in Figure 5. The user provides a PLC variable manipulation goal as input to ReVISE. The goal is uniformly defined as a tuple $\langle C, V, x, N \rangle$, meaning that:

- When a condition C involving PLC variable(s) holds, ReVISE should change the target variable V ’s value to x (the maliciously intended value), and repeat this manipulation N times.

We note that the manipulation goal can be generated using existing PLC attack planning tools (e.g., [9], [10]), *without* requiring the user to have in-depth knowledge about the target ICS.

TABLE II: PLC variable snapshots in the water treatment plant.

Request ID	FIT201 (S)	P203 (A)	FIT201 (S)	P203 (A)
1 (w/ two snapshots)	0.25	0.10	0.28	0.10
2 (w/ two snapshots)	0.42	0.10	0.42	0.20

S: sensing variable; A: actuation variable.

ReVISE will then automatically carry out the requested manipulation by performing the following three main steps (Figure 6):

(1) *Variable Value Monitoring (Section V-B)*: ReVISE first passively monitors PLC variable values via a benign server program running concurrently with the main PLC program. Specifically, ReVISE leverages the (legitimate) client-side API of that server program to request and acquire sequential “snapshots” of PLC variable values, which will be checked against the condition C in the manipulation goal.

(2) *Timing-aware Variable Manipulation (Section V-C)*: When the condition C holds, ReVISE launches the actual manipulation of the target variable (value write), in synchronization with the execution of the main PLC program³, based on variable value snapshots obtained. The goal is to increase the probability that the manipulation happens between the benign update and usage of the target variable.

(3) *Original Variable Value Restoration (Section V-D)*: After the manipulated variable value is used by the main PLC program, ReVISE restores the last-obtained original value of the target variable to maintain stealth. To determine the timing of such restoration, ReVISE will request an extra couple of variable value snapshots to confirm that the main PLC program has consumed the manipulated variable value.

B. Step 1: Variable Value Monitoring

In this step, ReVISE passively monitors the target PLC’s variable values to find an appropriate timing for remotely launching the requested variable manipulation.

Polling PLC variable values. ReVISE invokes (legitimate) client-side API of a server program in PLC to request and acquire a sequence of “snapshots” of PLC variable values. Each snapshot is a set of PLC variable values with a timestamp indicating when the request was sent to the PLC. ReVISE stores the snapshots in $Snapshot_{set}$.

ReVISE requests two snapshots in one client request, leveraging the “multiple reads of a variable in one request” feature of the server application (described in Section III-B). The server’s response contains two sequential snapshots taken in the same PLC scan cycle or in two consecutive cycles, allowing ReVISE to detect a variable value change, which indicates an update to the variable, either by the main PLC program (in the case of an actuation variable) or by a sensor (in the case of a sensing variable). For example, Table II shows four variable snapshots (two per row) from the water treatment plant, in response to two read requests. Each snapshot contains a sensing variable and an actuation variable. The two snapshots in the first row of Table II show a sensing variable value change

³The synchronization is not precise, due to the end-to-end latency between ReVISE and the main PLC program.

(from 0.25 to 0.28) hence indicating a sensor reading refresh between two consecutive cycles; whereas the two snapshots in the second row of Table II show an actuation variable value change (from 0.10 to 0.20) hence indicating the main PLC program's control output generation during the same cycle. ReVISE performs such polling at an interval of T , where T is a configurable parameter of ReVISE. We set the default value of T to 100 ms to imitate the polling frequency of benign clients of the embedded servers [30].

Goal-aware predictive monitoring. To poll PLC variable values when the likelihood of satisfying the manipulation condition C is high, ReVISE analyzes previously collected snapshots along with the manipulation goal G . Specifically, ReVISE uses the prediction function f_G to estimate the time for taking the next snapshot. Although variable value prediction techniques are commonly applied in ICS defenses [31], [32], ReVISE uses f_G offensively to avoid unnecessary polling.

For *threshold-based variable conditions* (e.g., $v > \theta$), f_G performs linear estimation on recent variable values to forecast when the threshold will be crossed. For *equality-based variable conditions* (e.g., $v = 1$), it analyzes recent toggling behavior to estimate the next expected transition time based on observed switching intervals. In the case of *multi-variable conjunctive conditions* (e.g., $v_1 > \theta_1 \wedge v_2 = \theta_2$), ReVISE independently predicts the satisfaction time for each sub-condition and selects the latest time.

C. Step 2: Timing-aware Variable Manipulation

In this step, ReVISE determines the timing to launch the requested variable manipulation. Specifically, it tries to catch the legitimate value change of the target variable at the PLC and synchronize the manipulation with the change.

Actuation variable manipulation. When ReVISE receives a pair of variable value snapshots (Step 1), it will first determine if the pre-condition C of the manipulation is satisfied. C is typically expressed as a value range that a PLC variable (not necessarily the variable being manipulated) may fall into. If the condition holds, ReVISE will further check if there is a change in the value of the target actuation variable between the two snapshots. If so, ReVISE will immediately launch the manipulation by submitting a write request to the in-PLC server application. This is because the observed value change is an indication that the main PLC program has generated its control output in the current cycle, which will remain the same until the end of the cycle (when the actuation variable value will be transmitted to the ICS actuator). As such, launching the manipulation right after the (legitimate) value change will maximize the probability that the manipulated value is transmitted to the actuator.

The first half of Algorithm 1 outlines ReVISE's logic for the actuation variable manipulation. The algorithm first checks if the most recent pair of snapshots is taken during the same scan cycle, based on the fact that the values of the sensor variables remain the same within the same cycle. Specifically, it first checks whether the sensing variable values are identical (Line 8). It then checks the actuation variable in the snapshots

Algorithm 1 ReVISE: PLC Variable Manipulation

Input: PLC variable snapshots (Snapshot_set), Condition (C), manipulation value x , attack times N

Output: PLC variable manipulated

```

1: count ← 0
2: for each snapshot in Snapshot_set do
3:   prevSensor ← snapshot.PrevSensorValue
4:   currSensor ← snapshot.CurrSensorValue
5:   prevAct ← snapshot.PrevActuationValue
6:   currAct ← snapshot.CurrActuationValue
   Case 1: Manipulating an actuation variable
7:   if isValid( $C$ ) then
8:     if prevSensor = currSensor then
9:       if prevAct ≠ currAct then
10:        LaunchManipulation( $x$ )
11:        RestoreOriginalValue(prevAct)
12:        count ← count + 1
13:        if count ≥  $N$  then
14:          break
15:        end if
16:      end if
17:    end if
18:  end if
   Case 2: Manipulating a sensing variable
19:  if isValid( $C$ ) then
20:    if prevSensor ≠ currSensor then
21:      if prevAct = currAct then
22:        LaunchManipulation( $x$ )
23:        RestoreOriginalValue(prevSensor)
24:        count ← count + 1
25:        if count ≥  $N$  then
26:          break
27:        end if
28:      end if
29:    end if
30:  end if
31:  if count ≥  $N$  then
32:    break
33:  end if
34: end for

```

to detect a change in its value (Line 9). For example, ReVISE confirms that the two snapshots in the *second* row of Table II belong to the same PLC scan cycle, with the sensing variable value remaining the same (0.42), whereas the actuation variable value changes from 0.10 to 0.20. ReVISE immediately launches the remote manipulation, changing the actuation variable's value to the intended value of x (Line 10).

Sensing variable manipulation. For sensing variable manipulation, ReVISE will check if (1) the pre-condition C is satisfied with the new value; (2) there is a change of value of the target sensing variable between the two snapshots; and (3) the actuation variable values remain the same in the two snapshots in the snapshot pair. If (1)-(3) are all true, ReVISE will immediately launch the manipulation by submitting a write request to the in-PLC server application. This is because the observed sensing variable value change, together with the unchanged actuation variable value, is an indication that the sensing variable value has been updated with the sensor's reading *across two consecutive cycles*. And launching the manipulation right away will maximize the probability that the manipulated value will be used by the main PLC program.

The second half of Algorithm 1 outlines ReVISE's logic for manipulating a sensing variable. Specifically, it checks if the pair of snapshots belongs to two consecutive scan cycles (i.e.,

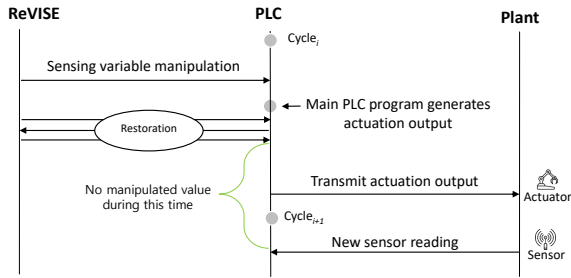


Fig. 7: Sensing variable restoration after manipulation.

with different sensing variable values (Line 20) and identical actuation variable values (Line 21). If so, ReVISE launches the sensing variable manipulation (Line 22). For example, the two snapshots in the first row of Table II indicate such a good timing for manipulating the sensing variable FIT201, with an identical actuation variable value (0.10) and a change of sensing variable value (from 0.25 to 0.28).

D. Step 3: Restoring Original Variable Value

In this step, ReVISE restores the manipulated variable to its most recent original value after a successful manipulation. This is to minimize the probability that the manipulated value will be seen by an external monitor or anomaly detector. We do note that, with the periodic execution of the main PLC program, any manipulated value will be overwritten with a benign value *in the next PLC scan cycle*, by either the sensor reading (for the sensor variable) or the main PLC program (for the actuation variable). Yet, our restoration is still necessary to “cover up” the manipulation before the next legitimate overwrite happens. **Sensing variable restoration.** ReVISE restores a manipulated sensing variable to its most recent benign value after the main PLC program has used the manipulated value to compute actuation output, as illustrated in Figure 7. More specifically, ReVISE performs one more PLC variable poll after the manipulation to check if the sensing variable still has the manipulated value, *and* the actuation variable value is new. If so, the main PLC program has generated the actuation output, indicating that the manipulated variable can be restored. ReVISE then restores the sensing variable with its most recent value before manipulation, by sending a write request.

Actuation variable restoration. ReVISE restores a manipulated actuation variable to its most recent benign value, after the manipulated value has been transmitted to the physical actuator at the end of the current cycle. More specifically, ReVISE performs one more variable poll after the manipulation. If the actuation variable is still the manipulated value *and* the sensing variable value is new, it can be inferred that the PLC has sent the manipulated actuation variable value to the actuator, and a new cycle has started. As such, ReVISE immediately restores the actuation variable to its benign value before manipulation.

VI. IMPLEMENTATION

We have developed the ReVISE tool in C# with four key modules: (1) A user-friendly interface (GUI) that allows the user to select the PLC variable manipulation goal. (2)

A monitoring module that remotely polls the PLC variable values via (legitimate) client-side API of a server program in PLC (currently supporting OPC UA and web server). (3) A timing-aware PLC variable manipulation module that leverages the observed variable values to determine the timing of and launch remote variable manipulations. (4) A variable value restoration module that restores the manipulated PLC variable to its original benign value after a successful manipulation. The ReVISE tool supports three commodity PLCs from Siemens, CodeSys, and Unipi, respectively.

To demonstrate seamless interfacing with existing ICS attack generation tools (to define PLC variable manipulation goals for ReVISE), we also re-implemented two such tools, AttackLLM [9] and SmartFuzz [10], which require no in-depth knowledge about the target ICS process. To ensure the effectiveness of our implementations, we confirmed that our implementations are able to generate the same attack tactics (i.e., which PLC variable to manipulate and by how much) as those reported in [9] and [10].

VII. EVALUATION

A. Experimental Setup

We evaluate the effectiveness of ReVISE using 15 remote PLC variable manipulation attacks on three ICS platforms with real PLCs in the loop: the Fischertechnik manufacturing mini plant [17] controlled by a Siemens S7 1500 PLC [33], a simulated chemical plant [18] controlled by a Unipi PLC [34], and a simulated water treatment plant [19] controlled by a CodeSys PLC [35]. Previous works have also used these platforms to identify vulnerabilities and evaluate attacks and defenses in ICS [2], [36], [37].

Remote PLC variable manipulation Attacks. The 15 remote PLC variable manipulation attacks on the three evaluation ICS platforms are shown in Table III. These attacks are generated by either AttackLLM or SmartFuzz, without requiring detailed knowledge about the target platforms. Table IV shows the attacks’ PLC variable manipulation goals, which are easily derived from the attack generation tools’ outputs. For example, in Attack *FP1*, its PLC variable manipulation goal as follows:

IF $80 \leq VGR_Sensor \leq 100$ then $VGR_Sensor = 601$ for 1 time.

This goal reflects the “plot” of the first attack in Section IV, which is to cause the VGR robot to drop a workpiece onto an unsafe location by manipulating sensing variable *VGR_Sensor*.

Given the manipulation goals, ReVISE automatically performs the requested manipulations in alignment with our threat model (Section II). These attacks also align with the MITRE ATT&CK techniques for ICS [38], such as Modify Parameter (T817) and Unauthorized Command Message (T1047). As such, we believe that our attacks are representative and portable to other ICS/PLC environments.

The machine we use for running ReVISE (i.e., the attacker’s device) is a commodity PC equipped with Windows 11 (64-bit) OS, 16 GB of memory, and an Intel(R) Core(TM) i7-8565U processor operating at 3.79 GHz. The network between the attacker’s PC and the target PLC is a wired 100Mbps Ethernet.

TABLE III: Overview of 15 remote PLC variable manipulation attacks for evaluation.

ID	Description	Generation Tool	PLC
FP_1	Modify VGR sensing variable to before reaching target, causing workpiece to drop.	AttackLLM, SmartFuzz	Siemens
FP_2	Alter the NFC actuation variable after the main PLC program sets it to start.	AttackLLM, SmartFuzz	Siemens
FP_3	Reverse the conveyor belt when the PLC sets its actuation variable to move it forward.	AttackLLM	Siemens
FP_4	Shut down the MPO oven by manipulating the actuation variable after it started.	AttackLLM, SmartFuzz	Siemens
FP_5	Manipulate the horizontal movement of the VGR robot, overriding the main PLC program.	AttackLLM	Siemens
FP_6	Modify the MPO robot actuation variable to prematurely stop its movement.	AttackLLM	Siemens
FP_7	Stop VGR robot with actuation variable when sensing variable falls within a specific range.	AttackLLM	Siemens
FP_8	Manipulate the actuation variable of HBW when the PLC writes the benign value to stop it.	AttackLLM	Siemens
CP_1	Increase product flow sensing variable causes the PLC to limit the product valve.	AttackLLM, SmartFuzz	Unipi
CP_2	Manipulating the actuation variable to open the purge valve, degrading product quality.	AttackLLM	Unipi
CP_3	Increase pressure sensing variable after real-time data updates, decreases pressure level.	AttackLLM, SmartFuzz	Unipi
CP_4	Manipulating product valve actuation variable after main PLC program updates.	AttackLLM	Unipi
WP_1	Increasing water input causing malicious adjustments to HCl, NaOCl, and NaCl valves.	AttackLLM	CodeSys
WP_2	Manipulate actuation variable of NaOCl after PLC writes a benign value.	AttackLLM, SmartFuzz	CodeSys
WP_3	Decrease the sensing variable value when the real-time water input within a specific range.	AttackLLM	CodeSys

Testbeds: CP : chemical plant, FP : Fischertechnik manufacturing plant, WP : water treatment plant.

TABLE IV: PLC variable manipulation goals of the 15 attacks.

Attack ID	Manipulation Goal	Manipulated PLC Variable
FP_1	If $80 \leq VGR_Sensor \leq 100$ then $VGR_Sensor = 601$ for 1 time	$VGR_Sensor(S)$
FP_2	If $True$ then $NFC_Start = False$ for 1 time	$NFC_Start(A)$
FP_3	If $150 \leq HBW_Sensor \leq 200$ then $HBW_Conveyor = False$ for 1 time	$HBW_Conveyor(A)$
FP_4	If $True$ then $MPO_Oven = False$ for 1 time	$MPO_Oven(A)$
FP_5	If $Horizontal_Sensor \leq 10$ then $Horizontal_Sensor = 30$ for 1 time	$Horizontal_Sensor(S)$
FP_6	If $90 \leq MPO_Sensor \leq 120$ then $MPO_Actuator = False$ for 1 time	$MPO_Actuator(A)$
FP_7	If $0 \leq VGR_Sensor \leq 10$ then $VGR_Actuator = False$ for 1 time	$VGR_Actuator(A)$
FP_8	If $HBW_Actuator = False$ then $HBW_Actuator = True$ for 1 time	$HBW_Actuator(A)$
CP_1	If $74.33 \leq IW7 \leq 76.33$ then $IW7 = IW7 + 2.1$ for 10 times	$IW7(S)$
CP_2	If $50 \leq QW2 \leq 65$ then $QW2 = QW2 + 0.6$ for 2 times	$QW2(A)$
CP_3	If $3600 \leq IW8 \leq 3700$ then $IW8 = 3750$ for 1 times	$IW8(S)$
CP_4	If $50 \leq QW3 \leq 95$ then $QW3 = 45$ for 1 times	$QW3(A)$
WP_1	If $0.10 \leq FIT201 \leq 0.30$ then $FIT201 = FIT201 + 0.3$ for 1 times	$FIT201(S)$
WP_2	If $P203 = 0.10$ then $P203 = 0.2$ for 10 times	$P203(A)$
WP_3	If $0.50 \leq FIT201 \leq 0.75$ then $FIT201 = 0.42$ for 1 times	$FIT201(S)$

S : sensing variable, A : actuation variable.

Research Questions. We present our evaluation results that answer the following research questions:

RQ1 How effective is ReVISE in deciding the timing of PLC variable manipulation, hence improving attack success rate?

RQ2 How effective is ReVISE in bypassing ICS defenses?

RQ3 How long does it take for ReVISE to successfully execute a PLC variable manipulation?

B. ReVISE Effectiveness (RQ1)

To evaluate ReVISE’s effectiveness, we perform each of the PLC variable manipulation attacks (Table IV) 100 times, with varying attack start times (relative to the operation of the target ICS). We consider an attack successful only if all the manipulations in an attack are successful. To confirm the success of each manipulation, we collect logs from each plant (which is instrumented with a logging function for evaluation only) and check whether the main PLC program either (1) uses a manipulated sensor variable value to compute an actuation output, or (2) transmits a manipulated actuation variable value to the plant’s actuator.

We also compare each ReVISE-enabled attack with a corresponding baseline attack that does not leverage ReVISE. In the baseline attack, the attacker has the *same* manipulation goal, but executes the requested variable manipulation

immediately, without performing the three-step operations of ReVISE (Section V-A). We note that our baseline attacks are similar to PLC variable manipulation attacks proposed in prior research [8], [39], except that our baseline attacks are via clients of in-PLC server programs, not directly against the main PLC program.

Table V presents our results, which indicate that the ReVISE-enabled attacks achieve a significantly higher success rate than the corresponding baseline attacks. The average success rate among the 15 ReVISE-enabled attacks is 86%, compared with a 6% average success rate among the baseline attacks.

The baseline approach has a low success rate because it lacks the timing awareness provided by ReVISE, which very often results in the manipulated variable values being *overwritten*, hence failing to cause the intended attack impact on the ICS. For example, in Attack CP_1 , when the attacker manipulates the product sensing variable in the chemical plant without synchronization with the main PLC program execution, the manipulation occurs either before its value gets over-written by the sensor reading (too early) or after the main PLC program has already computed the actuation output (too late). As a result, the main PLC program cannot use the manipulated value, causing the attack to fail.

TABLE V: Comparison of attack success rates.

Method	FP ₁	FP ₂	FP ₃	FP ₄	FP ₅	FP ₆	FP ₇	FP ₈	CP ₁	CP ₂	CP ₃	CP ₄	WP ₁	WP ₂	WP ₃
Baseline	6%	8%	5%	6%	9%	7%	11%	9%	0%	3%	4%	5%	5%	0%	7%
ReVISE	91%	84%	87%	85%	84%	87%	92%	89%	83%	86%	87%	85%	89%	80%	86%

TABLE VI: Manipulated leak rate (polling interval: 100ms).

Method	FP ₁	FP ₂	FP ₃	FP ₄	FP ₅	FP ₆	FP ₇	FP ₈	CP ₁	CP ₂	CP ₃	CP ₄	WP ₁	WP ₂	WP ₃
w/o ReVISE	24%	21%	19%	22%	29%	19%	23%	21%	18%	20%	16%	21%	26%	19%	29%
with ReVISE	4%	6%	3%	5%	5%	6%	3%	8%	4%	7%	4%	6%	7%	8%	6%

TABLE VII: Attack detection rate (polling interval: 100ms).

Method	FP ₁	FP ₂	FP ₃	FP ₄	FP ₅	FP ₆	FP ₇	FP ₈	CP ₁	CP ₂	CP ₃	CP ₄	WP ₁	WP ₂	WP ₃
SAIN	0%	6%	3%	5%	0%	0%	3%	8%	0%	7%	0%	6%	7%	0%	6%
Ghaeini et al.	4%	6%	3%	0%	0%	0%	0%	8%	0%	0%	0%	6%	0%	0%	6%

TABLE VIII: Attack detection rate (polling interval: per cycle).

Method	FP ₁	FP ₂	FP ₃	FP ₄	FP ₅	FP ₆	FP ₇	FP ₈	CP ₁	CP ₂	CP ₃	CP ₄	WP ₁	WP ₂	WP ₃
SAIN	0%	23%	25%	29%	0%	0%	30%	27%	0%	33%	0%	35%	29%	0%	32%
Ghaeini et al.	26%	23%	25%	0%	0%	0%	0%	27%	0%	0%	0%	35%	0%	0%	32%

C. ReVISE Evasion (RQ2)

Evasion of periodic PLC variable monitoring. After a successful variable manipulation, ReVISE will restore the variable to its most recent benign value, preventing the variable’s manipulated value from being observed by an ICS monitor. For example, the remote monitoring system (HMI) in a typical ICS (including our three experimental platforms) polls the PLC variables every 100 milliseconds and presents the values to the human operator. Anomaly detection systems [40], [41] also poll the PLC variables at similar frequencies to perform detection functions such as invariant checks based on the variable values observed.

As such, it is desirable (from the attacker’s perspective) for ReVISE to minimize the probability that a manipulated variable value is observed by a monitor/detector that performs periodic variable polling. Table VI compares the “manipulation leak rate” of the baseline and ReVISE-enabled attacks, regardless of the attacks’ success/failure. The manipulation leak rate is defined as the percentage of manipulation attacks that have at least one manipulated variable value observed by a periodic (at 100ms interval) PLC variable monitor, among the 100 attack runs. Our results in Table VI indicate that ReVISE significantly reduces the manipulation leak rate (an average of 6% among all 15 attacks), compared with that of the baseline attacks (an average of 22%).

We note that ReVISE cannot fully eliminate manipulated variable value leaks, due to the “best-effort” nature of ReVISE’s variable restoration mechanism (Section V-D), which does not guarantee the timely restoration of all manipulated variables. On the other hand, we also point out that a leak of manipulated variable value does *not* necessarily lead to the detection of the manipulation attack by the monitoring/detection system. The detection of such an attack depends on the sensitivity of the detection policy/algorithm employed, which may still miss the attack – even given the manipulated variable value.

Evasion of ICS defenses. We now evaluate the stealth of ReVISE in the face of two state-of-the-art ICS intrusion

detection systems (IDS), SAIN [2] and Ghaeini et al. [20]. Both IDSes have achieved a high detection rate (close to 100%) against prior PLC variable manipulation attacks without the “cover-up” tactics of ReVISE. We adopted the open-source implementation of SAIN and re-implemented the technique by Ghaeini et al., and evaluated their effectiveness in detecting attacks presented in the corresponding papers [2], [20]. We are able to reproduce the high detection rates reported as a validation of our implementation/instantiation.

Table VII presents the detection rate of the two IDSes against each of the 15 ReVISE-enabled attacks (among the 100 runs), under the same PLC variable polling interval (100ms) as our earlier manipulation leak rate measurement. We observe that: (1) the detection rates are consistently low; and (2) for each attack, the detection rates by the IDSes (Table VII) are *no higher than* the manipulation leak rate (Table VI) of the same attack under the same polling frequency – this is true by nature, because the IDSes can only detect an attack, by checking the system property invariants, given the leaked manipulated variable values.

Table VIII presents the attack detection rate, under a more “vigilant” setup: The IDS will poll and check the PLC variable values – once every scan cycle (approximately 25ms). We note that such a high-frequency invocation of IDS is *not* common in real-world PLCs, due to the high system overhead incurred and the constrained processing capacity of PLCs. We observe that the detection rates by the two IDSes are higher than those under a 100ms polling interval, yet they are still significantly lower than the very high detection rate (close to 100%) against the traditional variable manipulation attacks. Our results highlight the stealth of ReVISE-enabled attacks because of their *intra-cycle* nature, where variable value manipulation *and* recovery happen within a cycle with high probability.

D. Attack Duration (RQ3)

Table IX shows the time (maximum, minimum, and average) needed to perform *one* PLC variable manipulation by ReVISE. Specifically, we measure the time between the submission

TABLE IX: Duration of a successful ReVISE manipulation.

Attack ID	Manipulation Duration (MM:SS:ms)		
	Minimum	Average	Maximum
FP_1	00:00:007	00:50:080	01:20:100
FP_2	00:00:008	01:21:325	02:08:002
FP_3	00:00:007	00:18:110	01:02:160
FP_4	00:00:009	01:05:733	02:00:344
FP_5	00:00:007	00:25:418	02:19:541
FP_6	00:00:007	00:38:255	01:05:120
FP_7	00:00:008	00:21:144	02:45:430
FP_8	00:00:009	00:41:312	01:36:307
CP_1	00:00:006	00:14:625	00:34:211
CP_2	00:00:007	00:31:904	00:56:617
CP_3	00:00:006	00:11:334	00:31:409
CP_4	00:00:008	00:16:155	00:39:114
WP_1	00:00:008	00:06:618	00:22:169
WP_2	00:00:009	00:39:700	01:03:125
WP_3	00:00:007	00:08:054	00:19:423

of a manipulation request and the time when one requested manipulation is successfully executed. Our measurements show that, on average, the minimum manipulation duration is 7.5 ms, the average manipulation duration is 31.3 seconds, and the maximum manipulation duration is 74.9 seconds.

We have further analyzed the reasons behind the variations between the durations required to perform the manipulations. We find that the bulk of the manipulation duration is for checking the manipulation pre-condition C and waiting for it to hold. We note that C is specific to the attack scenario and semantics. For example, Attack FP_2 aims to set the NFC actuation variable to False at the specific scan cycle when the main PLC program sets it to True. As this event occurs every *one minute* in the Fischertechnik mini-plant, ReVISE tends to stay in the variable polling mode (Step 1 of ReVISE) for a long time (compared with some of the other attacks), leading to a relatively long manipulation duration.

VIII. COUNTERMEASURES

We propose two countermeasures that can potentially protect PLCs against ReVISE-enabled attacks. First, PLC vendors can isolate the variables used by the main PLC program from those used by server programs and implement a PLC-based monitor that validates every request from the server programs before it can affect the main PLC program variables. However, isolating these variables may prevent benign applications from functioning properly and could affect the real-time performance requirements of the PLC – for example, preventing operators from sending manual commands to the PLC in time when needed. Second, the PLC runtime can be enhanced by validating the PLC variables to ensure their integrity throughout the PLC cycles. Yet, this approach would also increase computational cost and impact real-time PLC properties. In our future work, we will analyze the efficacy and cost-effectiveness of these measures against ReVISE-enabled attacks.

IX. RELATED WORK

Prior research has shown that attackers manipulate PLC variables by injecting malicious code that writes specific

variable values into the main PLC program [6], [13], [42], [43]. Another line of work has explored compromising PLC firmware through rootkits [14], [44], which operate at a lower level but often require elevated privileges or physical access. Another line of attack involves overwhelming PLCs with traffic to disrupt their operation, e.g., DoS attacks [45].

Recent research has shown that PLCs are vulnerable to attacks that compromise the web server embedded in the PLC and then remotely supply malicious values of the target PLC variables as input to the compromised web server [15]. Follow-up studies have discovered security vulnerabilities in third-party PLC server programs. One study identified that programs embedded in PLCs often are not security-hardened and have weak authentication, which allows attackers to remotely manipulate the PLC variables [5].

ReVISE distinguishes itself from these research efforts by *extending* and *generalizing* the threat vector of PLC variable manipulation by revealing intra-cycle variable manipulation through any embedded server application and *without* compromising any program in the PLC. More specifically, ReVISE enables *timing-aware* PLC variable manipulation by modifying the value of the target variable at a time favorable to the success of the manipulation.

CPS Defenses. Prior research has also proposed multiple methods for securing programmable logic controllers and other cyber-physical controllers via formal verification of control logic correctness and safety [46]–[49], reachability-based hybrid-system analysis for safe operating bounds [50]–[52], and stochastic/probabilistic modeling to address uncertainties in system dynamics [53]–[55]. At the same time, improvements in real-time scheduling [56]–[58] and resilient/secure control frameworks [59]–[61] have strengthened robustness against faults and adversarial influence.

X. CONCLUSION

This paper reveals remote intra-cycle variable manipulation attacks through *any* embedded server program and *without* compromising any program in the PLC. As the first effort to address the timing (un)awareness challenge in improving the success rate of remote variable manipulations, this paper present ReVISE, an automated penetration-testing tool enables controlled testing of intra-cycle variable-manipulation attacks on PLCs. ReVISE achieves a significant increase in attack success rates from 6% to 86% across 15 attacks, while significantly reducing detection rate by restoring original variable values after manipulation.

ACKNOWLEDGMENT

We thank the anonymous reviewers for their timely, valuable comments and suggestions. This research was partially funded by the National Science Foundation (NSF) through grants CNS-2144645 and IIS-2229876. Any opinions, findings, and conclusions in this paper are those of the authors and do not necessarily reflect the views of the sponsor.

REFERENCES

- [1] E. D. Knapp, *Industrial Network Security: Securing critical infrastructure networks for smart grid, SCADA, and other Industrial Control Systems*, 2024.
- [2] S. G. Abbas, M. O. Ozmen, A. Alsaheel, A. Khan, Z. B. Celik, and D. Xu, "Sain: Improving ics attack detection sensitivity via state-aware invariants," in *USENIX Security Symposium*, 2024.
- [3] T. Ji and X. Xu, "Exploring the integration of cloud manufacturing and cyber-physical systems in the era of industry 4.0—an opc ua approach," *Robotics and Computer-Integrated Manufacturing*, 2025.
- [4] CISA, "Rockwell automation controllogix plc vulnerabilities," <https://www.cisa.gov/news-events/ics-advisories/icsa-13-011-03>, 2024, [Online; accessed January 20, 2025].
- [5] T. Sasaki, A. Fujita, C. H. Ganán, M. van Eeten, K. Yoshioka, and T. Matsumoto, "Exposed infrastructures: Discovery, attacks and remediation of insecure ics remote management devices," in *IEEE Symposium on Security and Privacy (SP)*, 2022.
- [6] M. Baezner and P. Robin, "Stuxnet," ETH Zurich, Tech. Rep., 2017.
- [7] F. Robles and N. Perlroth, "Dangerous stuff": hackers tried to poison water supply of florida town," in *The New York Times*, 2021.
- [8] J. H. Castellanos, M. Ochoa, A. A. Cardenas, O. Arden, and J. Zhou, "Attfinder: Discovering attack vectors in plc programs using information flow analysis," in *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*, 2021.
- [9] C. M. Ahmed, "Attackllm: Llm-based attack pattern generation for an industrial control system," in *Proceedings of the 2nd International Workshop on Foundation Models for Cyber-Physical Systems & Internet of Things*, 2025.
- [10] Y. Chen, C. M. Poskitt, J. Sun, S. Adepu, and F. Zhang, "Learning-guided network fuzzing for testing cyber-physical system defences," in *34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2019.
- [11] C. Feng, T. Li, Z. Zhu, and D. Chana, "A deep learning-based framework for conducting stealthy attacks in industrial control systems," *arXiv preprint arXiv:1709.06397*, 2017.
- [12] M. A. Umer, C. M. Ahmed, M. T. Jilani, and A. P. Mathur, "Attack rules: an adversarial approach to generate attacks for industrial control systems using machine learning," in *Proceedings of the 2th Workshop on CPS&IoT Security and Privacy*, 2021.
- [13] H. Yoo, S. Kalle, J. Smith, and I. Ahmed, "Overshadow plc to detect remote control-logic injection attacks," in *Detection of Intrusions and Malware, and Vulnerability Assessment: 16th International Conference, DIMVA 2019, Gothenburg, Sweden, June 19–20, 2019, Proceedings 16*. Springer, 2019, pp. 109–132.
- [14] L. Garcia, F. Brasser, M. H. Cintuglu, A.-R. Sadeghi, O. A. Mohammed, and S. A. Zonouz, "Hey, my malware knows physics! attacking plcs with physical model aware rootkit," in *NDSS*, 2017.
- [15] R. Pickren, T. Shekari, S. Zonouz, and R. Beyah, "Compromising industrial processes using web-based programmable logic controller malware," in *Network and Distributed System Security Symposium (NDSS)*, 2024.
- [16] C. Finck and T. Dohrmann, "A decade after stuxnet: How siemens s7 is still an attacker's heaven."
- [17] FischerTechnik, "Fischertechnik plant manual," https://www.fischertechnik.de/-/media/fischertechnik/fite/service/elearning/simulieren/lernfabrik-4-0-24v/fabrik_2020_deutsch_s7-1500_en_korrigiert_final.ashx, 2022, [Online; accessed January 15, 2025].
- [18] GRFICSv2, "Chemical plant," <https://github.com/Fortiphyd/GRFICSv2>, 2022, [Online; accessed January 15, 2025].
- [19] YuqiChen94, "Water treatment plant," https://github.com/yuqiChen94/Swat_Simulator, 2020, [Online; accessed January 15, 2025].
- [20] H. R. Ghaeni, D. Antonioli, F. Brasser, A.-R. Sadeghi, and N. O. Tippenhauer, "State-aware anomaly detection for industrial control systems," in *ACM Symposium on Applied Computing*, 2018.
- [21] M. Ike, K. Phan, K. Sadoski, R. Valme, and W. Lee, "Scaphy: Detecting modern ics attacks by correlating behaviors in scada and physical," in *IEEE Symposium on Security and Privacy (SP)*, 2023.
- [22] C. Irvine, T. Shekari, D. Formby, and R. Beyah, "If i knew then what i know now: On reevaluating dnp3 security using power substation traffic," in *Proceedings of the Fifth Annual Industrial Control System Security (ICSS) Workshop*, 2019.
- [23] CISA, "Apt cyber tools targeting ics/scada devices," <https://www.cisa.gov/news-events/cybersecurity-advisories/aa22-103a>, 2022, [Online; accessed March 20, 2025].
- [24] A. Ayub, H. Yoo, and I. Ahmed, "Empirical study of plc authentication protocols in industrial control systems. in 2021 ieee security and privacy workshops (spw)." 2021.
- [25] J. Hajda, R. Jakuszewski, and S. Ogonowski, "Security challenges in industry 4.0 plc systems," *Applied Sciences*, 2021.
- [26] A. Mirian, Z. Ma, D. Adrian, M. Tischer, T. Chuenchujit, T. Yardley, R. Berthier, J. Mason, Z. Durumeric, J. A. Halderman *et al.*, "An internet-wide view of ics devices," in *14th Annual Conference on Privacy, Security and Trust (PST)*, 2016.
- [27] A. Hassanzadeh, A. Rasekh, S. Galelli, M. Aghashahi, R. Taormina, A. Ostfeld, and M. K. Banks, "A review of cybersecurity incidents in the water sector," *Journal of Environmental Engineering*, 2020.
- [28] A. Abbasi, M. Hashemi, E. Zambon, and S. Etalle, "Stealth low-level manipulation of programmable logic controllers i/o by pin control exploitation," in *International Conference on Critical Information Infrastructures Security*, 2016.
- [29] CISA, "Idem plcs," <https://www.cisa.gov/news-events/ics-advisories/icsa-22-006-03>, 2022, [Online; accessed March 20, 2025].
- [30] O. Sande and M. Fojcik, "Opc ua communication traffic control for analogue values in an automation device with embedded opc ua servers," in *International Conference on Computational Collective Intelligence*, 2018.
- [31] F. Turrin, A. Erba, N. O. Tippenhauer, and M. Conti, "A statistical analysis framework for ics process datasets," in *Proceedings of the Joint Workshop on CPS&IoT Security and Privacy*, 2020.
- [32] P. von Stackelberg, R. Goedhart, Ş. I. Birbil, and R. J. Does, "Comparison of threshold tuning methods for predictive monitoring," *Quality and Reliability Engineering International*, 2024.
- [33] Siemens, "Siemens firmware update s7-1500 cpus," [https://support.industry.siemens.com/cs/document/109478459/firmware-update-s7-1500-cpus-incl-displays-and-et-200-cpus-\(et-200sp-et-200pro\)?dti=0&lc=en-DE](https://support.industry.siemens.com/cs/document/109478459/firmware-update-s7-1500-cpus-incl-displays-and-et-200-cpus-(et-200sp-et-200pro)?dti=0&lc=en-DE), 2024, accessed: January 15, 2025.
- [34] U. Technology, "Unipi neuron," https://www.unipi.technology/?gad_source=1&gclid=EAIaIQobChMIha3lqJ6EhwMV1zYIBR3LnQXdEAAYASAAEgL9_vD_BwE, 2024, [Online; accessed January 15, 2025].
- [35] "Codesys webvisu," <https://www.codesys.com/products/codesys-visualization/webvisu.html>, 2024, [Online; accessed March 15, 2025].
- [36] M. Zhang, C.-Y. Chen, B.-C. Kao, Y. Qamsane, Y. Shao, Y. Lin, E. Shi, S. Mohan, K. Barton, J. Moyne *et al.*, "Towards automated safety vetting of plc code in real-world plants," in *IEEE Symposium on Security and Privacy (SP)*, 2019.
- [37] S. Adepu and A. Mathur, "Generalized attacker and attack models for cyber physical systems," in *IEEE 40th annual computer software and applications conference (COMPSAC)*, 2016.
- [38] Mitre, "Ics attack techniques," <https://attack.mitre.org/techniques/ics/>, 2023, accessed: Jan 15, 2025.
- [39] B. Green, R. Derbyshire, M. Krotofil, W. Knowles, D. Prince, and N. Suri, "Pcaad: Towards automated determination and exploitation of industrial systems," *Computers & Security*, 2021.
- [40] W. Jardine, S. Frey, B. Green, and A. Rashid, "Senami: Selective non-invasive active monitoring for ics intrusion detection," in *Proceedings of the 2nd ACM Workshop on Cyber-Physical Systems Security and Privacy*, 2016.
- [41] E. J. Colbert and S. Hutchinson, "Intrusion detection in industrial control systems," *Cyber-security of SCADA and other industrial control systems*, 2016.
- [42] S. McLaughlin, "On dynamic malware payloads aimed at programmable logic controllers," in *6th USENIX Workshop on Hot Topics in Security (HotSec 11)*, 2011.
- [43] S. McLaughlin and P. McDaniel, "Sabot: specification-based payload generation for programmable logic controllers," in *Proceedings of the 2012 ACM conference on Computer and communications security*, 2012.
- [44] A. Abbasi, "Ghost in the plc: stealth on-the-fly manipulation of programmable logic controllers' i/o," in *Centre for Telematics and Information Technology (CTIT)*, 2016.
- [45] E. N. Ylmaz, B. Ciyilan, S. Gönen, E. Sindiren, and G. Karacayilmaz, "Cyber security in industrial control systems: Analysis of dos attacks against plcs and the insider effect," in *6th international istanbul smart grids and cities congress and fair (icsg)*, 2018.
- [46] S. Mitra, T. Wongpiromsam, and R. M. Murray, "Verifying cyber-physical interactions in safety-critical systems," *IEEE Security & Privacy*, 2013.

- [47] A. Abate, J.-P. Katoen, J. Lygeros, and M. Prandini, "Approximate model checking of stochastic hybrid systems," *European Journal of Control*, 2010.
- [48] S. Schupp, J. Nellen, and E. Ábrahám, "Divide and conquer: variable set separation in hybrid systems reachability analysis," *arXiv preprint arXiv:1707.04851*, 2017.
- [49] M. Althoff, M. Forets, Y. Li, S. Mitra, C. Schilling, M. Wetzlinger, and D. Zhuang, "Arch-comp23 category report: Continuous and hybrid systems with linear continuous dynamics," in *10th International Workshop on Applied Verification of Continuous and Hybrid Systems*, 2023.
- [50] R. Alur, T. Dang, and F. Ivančić, "Reachability analysis of hybrid systems via predicate abstraction," in *International Workshop on Hybrid Systems: Computation and Control*, 2002.
- [51] E. Abraham, J. Nellen, and S. Schupp, "Techniques and tools for hybrid systems reachability analysis," *Rigorous Systems Engineering of Cyber Physical Systems, RISE4CPS*, 2017.
- [52] G. Frehse, C. Le Guernic, A. Donzé, S. Cotton, R. Ray, O. Lebeltel, R. Ripado, A. Girard, T. Dang, and O. Maler, "Spaceex: Scalable verification of hybrid systems," in *International Conference on Computer Aided Verification*, 2011.
- [53] A. Abate, M. Prandini, J. Lygeros, and S. Sastry, "Probabilistic reachability and safety for controlled discrete time stochastic hybrid systems," *Automatica*, 2008.
- [54] M. Fränzle, E. M. Hahn, H. Hermanns, N. Wolovick, and L. Zhang, "Measurability and safety verification for stochastic hybrid systems," in *Proceedings of the 14th international conference on Hybrid systems: computation and control*, 2011.
- [55] A. Lavaei, S. Soudjani, and M. Zamani, "Compositional abstraction of large-scale stochastic systems: A relaxed dissipativity approach," *Nonlinear Analysis: Hybrid Systems*, 2020.
- [56] D. Casini, A. Biondi, G. Cicero, and G. Buttazzo, "Latency analysis of i/o virtualization techniques in hypervisor-based real-time systems," in *IEEE 27th Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2021.
- [57] A. Easwaran, "Demand-based scheduling of mixed-criticality sporadic tasks on one processor," in *IEEE 34th Real-Time Systems Symposium*, 2013.
- [58] M. Maggio, A. Hamann, E. Mayer-John, and D. Ziegenbein, "Control-system stability under consecutive deadline misses constraints," in *32nd euromicro conference on real-time systems*, 2020.
- [59] F. Miao, Q. Zhu, M. Pajic, and G. J. Pappas, "Coding schemes for securing cyber-physical systems against stealthy data injection attacks," *IEEE Transactions on Control of Network Systems*, 2016.
- [60] Y. Shoukry, P. Nuzzo, A. Puggelli, A. L. Sangiovanni-Vincentelli, S. A. Seshia, and P. Tabuada, "Secure state estimation for cyber-physical systems under sensor attacks: A satisfiability modulo theory approach," *IEEE Transactions on Automatic Control*, 2017.
- [61] Z. Liu and N. Ozay, "On the value of preview information for safety control," in *American Control Conference (ACC)*, 2021.